

BottleNet++: An End-to-End Approach for Feature Compression in Device-Edge Co-Inference Systems

Jiawei Shao, Jun Zhang

Department of Electronic and Information Engineering

The Hong Kong Polytechnic University, Hong Kong

Email: jiawei.shao@connect.polyu.hk, jun-eie.zhang@polyu.edu.hk

Abstract—The emergence of various intelligent mobile applications demands the deployment of powerful deep learning models at resource-constrained mobile devices. The device-edge co-inference framework provides a promising solution by splitting a neural network at a mobile device and an edge computing server. In order to balance the on-device computation and the communication overhead, the splitting point needs to be carefully picked, while the intermediate feature needs to be compressed before transmission. Existing studies decoupled the design of model splitting, feature compression, and communication, which may lead to excessive resource consumption of the mobile device. In this paper, we introduce an end-to-end architecture, named BottleNet++, that consists of an encoder, a non-trainable channel layer, and a decoder for more efficient feature compression and transmission. The encoder and decoder essentially implement joint source-channel coding via lightweight convolutional neural networks (CNNs), while explicitly considering the effect of channel noise. By exploiting the strong sparsity and the fault-tolerant property of the intermediate feature in deep neural networks (DNNs), BottleNet++ achieves a much higher compression ratio than existing methods. Compared with directly transmitting intermediate data without feature compression, BottleNet++ achieves up to $64\times$ bandwidth reduction over the additive white Gaussian noise channel and up to $256\times$ bit compression ratio in the binary erasure channel, with less than 2% reduction in accuracy of classification.

Index Terms—Deep Learning, Device-Edge Co-Inference, Network Compression, Joint Source-Channel Coding

I. INTRODUCTION

Recently, applications enabled by various mobile and Internet of Things (IoT) devices have profoundly changed our daily life [1]. One primary driver of these applications is the recent breakthrough in Deep Neural Networks (DNNs) [2] that for reliable inference. Unfortunately, DNN-based applications typically require a tremendous amount of computation, so they cannot be directly deployed on resource-constrained mobile/edge devices. A common method to solve this problem is to transmit the raw data to be processed at the cloud or edge computing platforms [3], [4]. The main disadvantage of this approach is the huge amount of communication overhead, which leads to considerable latency and energy consumption [5]. Another approach is on-device inference, which deploys compressed DNNs on mobile devices. However, the over-compressed networks cause severe performance degradation, and the limited resources lead to high on-device communication latency. These difficulties have driven the development of other alternatives, among which *device-edge co-inference* is a

promising solution [6], [7]. This method splits a network into the front part on a mobile device and the remaining part on an edge server. The output of the front part network is the intermediate feature, which would be transmitted to the edge server for further processing. In this way, device-edge co-inference achieves a good balance among on-device computation and the communication overhead, and thus reduces the inference latency.

Existing works on device-edge co-inference mainly investigated model splitting [8], while intermediate feature compression received less attention. Although DNNs can gradually abstract the intermediate feature layer by layer, the feature dimension may not decrease. There is an in-layer data amplification phenomenon [9], i.e., the output data size of early layers may be larger than the original input data [8]. Thus, without effective feature compression, the model splitting point needs to be deep enough until the size of the intermediate feature is small enough, which leads to more layers deployed on the device and more on-device computation. So the effectiveness of feature compression affects both the communication overhead and the amount of on-device computation.

In this paper, we propose an end-to-end trainable architecture, named BottleNet++, for efficient feature compression in *device-edge co-inference* systems. BottleNet++ consists of an encoder, a non-trainable channel layer, and a decoder. Both the encoder and decoder adopt lightweight convolutional neural networks (CNNs), which effectively act as joint source-channel coding [10]. Furthermore, by training the encoder and decoder in an end-to-end manner and explicitly considering the communication channel effect, BottleNet++ effectively exploits the fault-tolerant property of the DNN for a higher compression ratio. To further improve the model generalization ability in different channel conditions, the parameters of channel condition are considered by the encoder.

The main contributions of this paper are as follows:

- We introduce an end-to-end trainable architecture, named BottleNet++, for efficient intermediate feature compression and transmission for device-edge co-inference.
- We design an encoder that can perform adaptive coding under different channel conditions, which is robust to the varying channel conditions and achieves graceful degradation of accuracy with a noisy channel.
- Simulation results show that BottleNet++ achieves, with less than 2% accuracy degradation, up to $64\times$ bandwidth

BottleNet++: 设备边缘协同推理系统中特征压缩的端到端方法

邵家伟, 张军 香港理工大学电子及信息
工程系, 香港 电子邮件: jiawei.shao@connect.polyu.hk, ju
n-eie.zhang@polyu.edu.hk

Abstract—各种智能移动应用的出现要求在资源有限的移动设备上部署强大的深度学习模型。设备边缘协同推理框架通过在移动设备和边缘计算服务器上拆分神经网络, 提供了一种有前途的解决方案。为了平衡设备上的计算和通信开销, 需要仔细挑选分裂点, 而中间特征需要在传输前进行压缩。现有研究解耦了模型分割、特征压缩和通信的设计, 这可能会导致移动设备的资源消耗过多。在本文中, 我们介绍了一种名为 BottleNet++ 的端到端架构, 它由编码器、不可训练的通道层和解码器组成, 用于更有效的特征压缩和传输。编码器和解码器本质上是通过轻量级卷积神经网络 (CNN) 实现联合源通道编码, 同时明确考虑通道噪声的影响。通过利用深度神经网络 (DNN) 中中间特征的强稀疏性和容错特性, BottleNet++ 实现了比现有方法更高的压缩比。与不进行特征压缩而直接传输中间数据相比, BottleNet++ 在加性高斯白噪声通道上实现了高达 $64\times$ 的带宽减少, 在二进制擦除通道中实现了高达 $256\times$ 位的压缩比, 分类精度降低了不到 2 %。

Index Terms—深度学习、设备边缘协同推理、网络压缩、联合源通道编码

一、简介

最近, 各种移动和物联网 (IoT) 设备支持的应用程序深刻地改变了我们的日常生活[1]。这些应用的主要驱动力之一是深度神经网络 (DNN) [2] 最近在可靠推理方面取得的突破。不幸的是, 基于 DNN 的应用程序通常需要大量计算, 因此它们无法直接部署在资源有限的移动/边缘设备上。解决这个问题的常用方法是将要处理的原始数据传输到云端或边缘计算平台[3]、[4]。这种方法的主要缺点是大量的通信开销, 这会导致相当大的延迟和能耗[5]。另一种方法是设备端推理, 即在移动设备上部署压缩的 DNN。然而, 过度压缩的网络会导致严重的性能下降, 并且有限的资源导致设备上的通信延迟较高。这些困难推动了其他替代方案的开发, 其中 *device-edge co-inference* 是一个

有希望的解决方案[6], [7]。该方法将网络分为移动设备上的前部和边缘服务器上的剩余部分。前端网络的输出是中间特征, 将被传输到边缘服务器进行进一步处理。通过这种方式, 设备边缘协同推理在设备端计算和通信开销之间取得了良好的平衡, 从而降低了推理延迟。

现有的设备边缘协同推理工作主要研究模型分裂[8], 而中间特征压缩受到的关注较少。尽管 DNN 可以逐渐逐层抽象中间特征, 但特征维度可能不会减少。存在层内数据放大现象[9], 即早期层的输出数据大小可能大于原始输入数据[8]。因此, 如果没有有效的特征压缩, 模型分裂点需要足够深, 直到中间特征的大小足够小, 这导致设备上部署更多的层和更多的设备上计算。因此, 特征压缩的有效性会影响通信开销和设备上的计算量。

在本文中, 我们提出了一种名为 BottleNet++ 的端到端可训练架构, 用于在 *device-edge co-inference* 系统中进行有效的特征压缩。BottleNet++ 由编码器、不可训练的通道层和解码器组成。编码器和解码器都采用轻量级卷积神经网络 (CNN), 它有效地充当联合源通道编码[10]。此外, 通过以端到端的方式训练编码器和解码器并明确考虑通信信道效应, BottleNet++ 有效地利用了 DNN 的容错特性来获得更高的压缩比。为了进一步提高模型在不同信道条件下的泛化能力, 编码器考虑了信道条件的参数。

本文的主要贡献如下:

- 我们引入了一种名为 BottleNet++ 的端到端可训练架构, 用于设备边缘协同推理的高效中间特征压缩和传输。
- 我们设计了一种可以在不同信道条件下执行自适应编码的编码器, 该编码器对不同的信道条件具有鲁棒性, 并且可以在噪声信道下实现准确度的适度降低。
- 仿真结果表明, BottleNet++ 在精度下降不到 2% 的情况下实现了高达 $64\times$ 的带宽

reduction on the additive white Gaussian noise (AWGN) channel and up to $256\times$ bit compression ratio on the binary erasure channel (BEC) compared with the direct feature transmission without compression. With a higher compression ratio, BottleNet++ enables splitting a DNN at earlier layers, which leads to up to $3\times$ reduction in on-device computation compared with other compression methods.

II. PRELIMINARY

DNN has become a powerful method for many applications, but it typically requires a tremendous amount of computation. To deploy a DNN on source-constrained devices for edge AI applications, we consider hybrid deployment [11], called device-edge co-inference in this paper, which utilizes both the mobile device and the edge server for the execution to achieve a better tradeoff between communication and on-device computation. Our study focuses on increasing the compression ratio of the intermediate feature, which can reduce the communication overhead and on-device computation. It is achieved by exploiting the compression potential from model compression, joint source-channel coding, as well as the fault-tolerant property of neural networks.

A. Network Splitting and Feature Compression

Network splitting and feature compression are two critical problems for device-edge co-inference. The selection of the splitting point needs to reduce the latency due to on-device communication and feature transmission simultaneously. Feature compression has attracted lots of recent attention. Many works have proposed different splitting and compression methods. The feature coding method proposed in [6] applied JPEG coding and Huffman coding to compress the intermediate data. The method in [12] combined network splitting and model pruning, which first prunes the weights of the network and then selects the splitting point based on the pruned model. However, the pruning step would consume plenty of time. More recently, BottleNet [13] proposed to encode the intermediate feature by a neural network and use a compression-aware training approach to reduce the accuracy loss. The name of our architecture, i.e., BottleNet++, is inspired by this work.

All of the works mentioned above only considered source coding, while assuming reliable communication over the wireless channel, i.e., they adopted the separate principle of source and channel coding. Moreover, their design objective is to recover the intermediate feature at the edge server, either perfectly (with lossless source coding) or with tolerable distortion (with lossy compression). But, for the device-edge co-inference systems, reliable communication is not necessary. In other words, over-compression and inaccurate communication can be tolerated as long as they do not seriously affect the inference performance. Motivated by the above discussion, we propose an end-to-end design approach based on joint source-channel coding, while exploiting the fault-tolerant property of DNNs. In the next two subsections, the two main ingredients, i.e., joint source-channel coding and the fault-tolerant property

of DNNs, are introduced. The proposed framework will be presented in Section III.

B. Joint Source-Channel Coding

According to Shannons source-channel separation theorem [14], it is optimal to separate the design of source coding and channel coding. However, the optimality of the separate design holds only in the asymptotic limit of infinitely long source and channel blocks. In practice, joint source-channel coding can achieve better performance [15]. Recently, many works have tried learning-based methods for joint source-channel coding. The work [10] considered using the DNN to encode and decode the image over the AWGN channel. Another work [16] implemented the joint source-channel coding on the text transmission over a binary erasure channel, which uses the sequence-to-sequence learning framework to encode and decode the text.

However, the above studies focused on restoring the transmitted message at the receiver, which is a challenging communication problem but is not fully aligned with the overall design objective in our considered problem. As DNNs enjoy a fault-tolerant property, the high reliability in intermediate feature transmission is not needed, which gives further room for compression. To the best of our knowledge, this is the first work that exploits this unique opportunity for feature compression.

C. Fault-Tolerance Property of Neural Networks

Fault tolerance is frequently cited as a key property of neural networks [17] since these networks contain more neurons or processing elements than necessary to solve a given problem. This property can be leveraged for reducing the communication overhead in the device-edge co-inference systems by relaxing the reliability requirement of transmission. For this purpose, we propose to add the channel effect directly to the DNN during the training process, to exploit its fault-tolerant capability. As to be shown in the experiments, thanks to the fault-tolerance property, even if the channel noise corrupts the transmitted data, the DNN performance degrades gracefully.

III. PROPOSED ARCHITECTURE

The proposed end-to-end architecture, namely, BottleNet++, is shown in the box of Fig. 1. It consists of an encoder, a non-trainable channel layer, and a decoder. BottleNet++ is deployed at the splitting point of DNN to compress and transmit the cubelike intermediate feature tensors. The encoder and decoder are a pair of complementary lightweight CNNs. The extra computation introduced by the encoder and decoder is negligible compared to the whole network. The wireless channel is modeled as a non-trainable layer in DNN and represented by a transfer function, similar to DNN based joint source-channel coding [10]. In this paper, we only consider two simple channel models, i.e., the AWGN and binary erasure channels. For different channel models, the transfer function can be replaced accordingly, and our BottleNet++ can still work due to the network fault tolerance.

与无压缩的直接特征传输相比, 加性高斯白噪声 (AWGN) 通道的减少以及二进制擦除通道 (BEC) 上高达 $256\times$ 位的压缩比。凭借更高的压缩比, BottleNet++ 能够在较早的层分割 DNN, 与其他压缩方法相比, 设备上的计算量最多可减少 $3\times$ 。

二. 初步的

DNN 已成为许多应用的强大方法, 但它通常需要大量计算。为了在边缘人工智能应用程序的源受限设备上部署 DNN, 我们考虑混合部署 [11], 在本文中称为设备边缘协同推理, 它利用移动设备和边缘服务器来执行, 以实现通信和设备上计算之间的更好权衡。我们的研究重点是提高中间特征的压缩比, 这可以减少通信开销和设备上计算。它是通过利用模型压缩、联合源通道编码以及神经网络的容错特性的压缩潜力来实现的。

A. Network Splitting and Feature Compression

网络分裂和特征压缩是设备边缘协同推理的两个关键问题。分裂点的选择需要同时减少设备端通信和特征传输带来的延迟。特征压缩最近引起了很多关注。许多作品提出了不同的分割和压缩方法。文献[6]提出的特征编码方法应用 JPEG 编码和 Huffman 编码来压缩中间数据。文献[12]中的方法将网络分裂和模型剪枝结合起来, 首先剪枝网络的权重, 然后根据剪枝后的模型选择分裂点。然而, 修剪步骤会消耗大量时间。最近, BottleNet [13] 提出通过神经网络对中间特征进行编码, 并使用压缩感知训练方法来减少准确性损失。我们的架构的名称, 即 BottleNet++, 就是受到这项工作的启发。

上述所有工作都只考虑了信源编码, 同时假设无线信道上的可靠通信, 即采用信源编码和信道编码分离的原则。此外, 他们的设计目标是在边缘服务器上完美地恢复中间特征 (使用无损源编码) 或具有可容忍的失真 (使用有损压缩)。但是, 对于设备边缘协同推理系统来说, 可靠的通信并不是必需的。换句话说, 只要不严重影响推理性能, 过度压缩和不准确的通信是可以容忍的。受上述讨论的启发, 我们提出了一种基于联合源通道编码的端到端设计方法, 同时利用 DNN 的容错特性。在接下来的两小节中, 两个主要成分, 即联合源通道编码和容错特性

介绍了 DNN。拟议的框架将在第三节中介绍。

B. Joint Source-Channel Coding

根据香农信源信道分离定理[14], 将信源编码和信道编码的设计分开是最优的。然而, 单独设计的最优性仅在无限长源和通道块的渐近极限下成立。在实践中, 联合源信道编码可以获得更好的性能[15]。最近, 许多工作尝试了基于学习的联合源通道编码方法。工作 [10] 考虑使用 DNN 通过 AWGN 通道对图像进行编码和解码。另一项工作 [16] 在二进制擦除通道上实现了文本传输的联合源通道编码, 它使用序列到序列学习框架对文本进行编码和解码。

然而, 上述研究侧重于在接收器处恢复传输的消息, 这是一个具有挑战性的通信问题, 但与我们考虑的问题的总体设计目标并不完全一致。由于 DNN 具有容错特性, 因此不需要中间特征传输的高可靠性, 这为进一步的压缩提供了空间。据我们所知, 这是第一个利用这一独特机会进行特征压缩的工作。

C. Fault-Tolerance Property of Neural Networks

容错性经常被认为是神经网络的关键属性[17], 因为这些网络包含的神经元或处理元素多于解决给定问题所需的数量。通过放宽传输的可靠性要求, 可以利用该属性来减少设备边缘协同推理系统中的通信开销。为此, 我们建议在训练过程中将通道效应直接添加到 DNN 中, 以利用其容错能力。实验表明, 由于容错特性, 即使通道噪声破坏了传输的数据, DNN 的性能也会适度下降。

三. 拟议的架构

所提出的端到端架构, 即 BottleNet++, 如图 1 的方框所示。它由编码器、不可训练的通道层和解码器组成。BottleNet++ 部署在 DNN 的分裂点, 用于压缩和传输立方体中间特征张量。编码器和解码器是一对互补的轻量级 CNN。与整个网络相比, 编码器和解码器引入的额外计算可以忽略不计。无线信道被建模为 DNN 中的不可训练层, 并由传递函数表示, 类似于基于 DNN 的联合源信道编码 [10]。在本文中, 我们只考虑两种简单的信道模型, 即 AWGN 和二进制擦除信道。对于不同的通道模型, 可以相应地替换传递函数, 并且由于网络容错, 我们的 BottleNet++ 仍然可以工作。

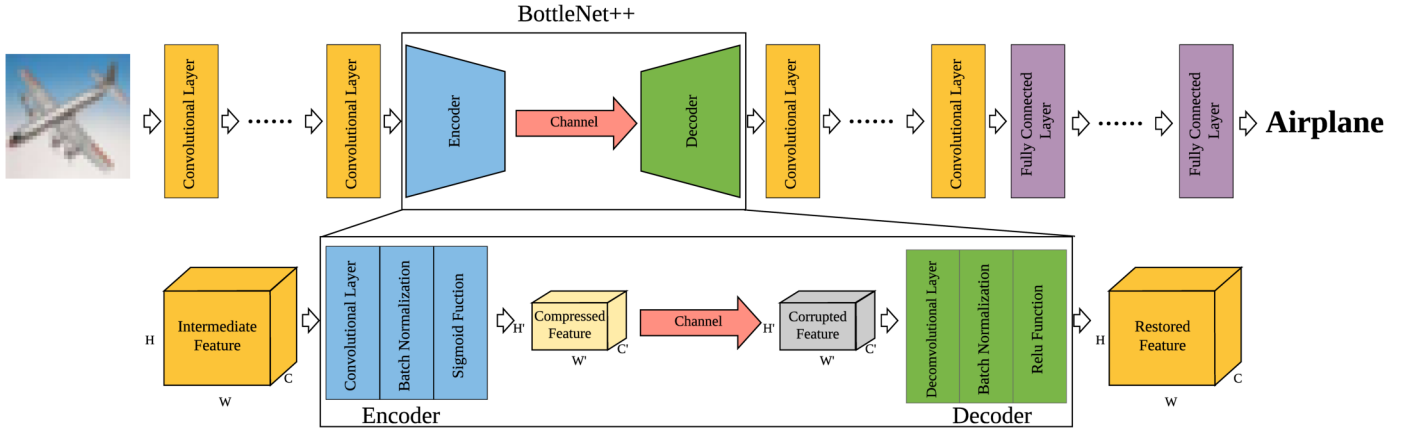


Fig. 1: The architecture of BottleNet++.

A. Encoder

The encoder plays the role of feature compression and joint source-channel coding, which consists of a convolutional layer, a batch normalization layer, and an activation layer. The encoder applies lossy compression to reduce the dimension of the intermediate feature of DNN. The convolutional layer uses different numbers of filters to control the output channel., and the stride of the convolutional operation and the kernel size determine the spatial size of the output feature. Concretely, the intermediate feature can be represented by a tensor as (channel, width, height). The four cubes presented in Fig. 1 are feature tensors with size (C, W, H) or (C', W', H') . To compress C channels to C' channels, the convolutional layer adopts C' filters. To realize width-wise reduction from W to W' and height-wise reduction from H to H' , the stride of convolutional operation is set to $(\lfloor W/W' \rfloor, \lfloor H/H' \rfloor)$. The convolutional layer is followed by a batch normalization layer and a Sigmoid activation function to add non-linearity features in the encoder. We use a Sigmoid function as the activation function because the output values are constrained to $[0, 1]$, which can be scaled to satisfy the transmitter output power constraint and further benefit to quantize data in the digital communication system. Furthermore, because the wireless channel is time-varying, in order to improve the encoder generalization ability, the encoder is designed to be adaptive to different channel conditions. Assuming the channel state information is available, we will add it as a parameter to the encoder.

B. Channel Model

After encoded, the compressed feature is sent through the ‘unreliable’, which would be corrupted by the channel noise. This process is shown at the bottom box of Fig. 1. We consider two different types of channel models. First we consider the AWGN model and the transfer function is written as $f(x) = x + n$, with $n \sim \mathcal{N}(0, \sigma^2)$. The parameter σ^2 captures the noise variance as the channel condition. The analog communication

scheme is adopted over the AWGN channel, i.e., symbols of the compressed feature will be directly modulated without digitizing or channel encoding. Besides, we also consider the binary erasure channel (BEC), which is also adopted in [16] for a DNN-based communication system. The BEC uses the bit erasure rate (BER) p to model the deep fades or burst errors in the channel model, which has binary input and ternary output. The value of the erasure bit is set to the average of the bit taking 0 and 1 in our experiment. For instance, converting a binary number to a decimal number, the value of 110 is 6. However, if the channel erases the leftmost bit, the receiver would assign the average of 010 and 110, which is 4, to it. For the BEC model, the continuous data must be quantized to n -bit string before transmission. The output of the Sigmoid function is constrained in $[0, 1]$, so the quantizer is $\tilde{X} = \text{round}(X \cdot (2^n - 1)) / (2^n - 1)$, where the float-point X is quantized to $\tilde{X}$, presented by n -bit sequence.

The AWGN model can be used directly in the end-to-end training because the transfer function is differentiable. However, the transfer function of the BEC and the quantization are non-differentiable, hindering the back-propagation process. To solve this problem, we simply skip the channel model in the back-propagation. In the experiment, we find that ignoring the quantization and channel corruption is feasible, and the model still converges when the value of p is not too large.

C. Decoder

The decoder deployed at the receiver is a joint source-channel decoder to map the corrupted feature to the restored feature. It consists of a deconvolutional layer, a batch normalization layer, and a ReLU activation function. The decoder aims to restore the bitstream to the feature tensor with the same dimension before compression. The number of filters in the deconvolutional layer determines the output channel number. To recover C' channels from C channels, the deconvolutional layer use C filters. Width-wise and height-wise restoration can adopt $(\lfloor W/W' \rfloor, \lfloor H/H' \rfloor)$ stride in convolutional operation,

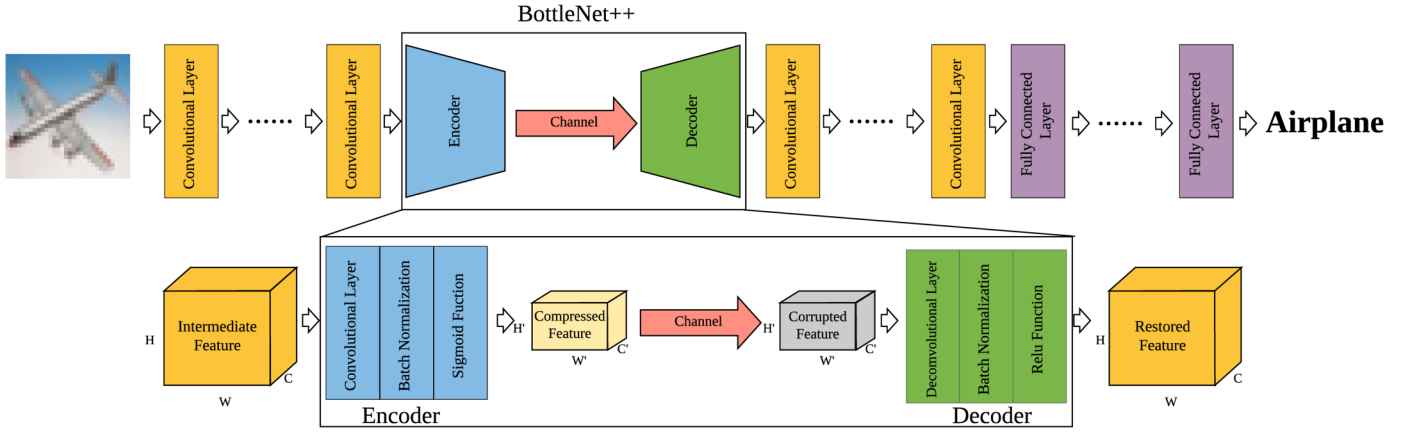


图 1: BottleNet++ 的架构。

A. Encoder

编码器起到特征压缩和联合源通道编码的作用，由卷积层、批量归一化层和激活层组成。编码器应用有损压缩来降低 DNN 中间特征的维度。卷积层使用不同数量的滤波器来控制输出通道，卷积运算的步长和内核大小决定了输出特征的空间大小。具体地，中间特征可以用张量表示为(通道，宽度，高度)。图 1 中呈现的四个立方体是大小为 (C, W, H) 或 (C', W', H') 的特征张量。为了将 C 通道压缩为 C' 通道，卷积层采用 C' 滤波器。为了实现从 W 到 W' 的宽度方向减少以及从 H 到 H' 的高度方向减少，卷积运算的步长被设置为 $\lfloor W/W' \rfloor, \lfloor H/H' \rfloor$ 。卷积层后面是批量归一化层和 Sigmoid 激活函数，用于在编码器中添加非线性特征。我们使用 Sigmoid 函数作为激活函数，因为输出值被限制为 $[0, 1]$ ，可以缩放以满足发射机输出功率限制，并进一步有利于量化数字通信系统中的数据。此外，由于无线信道是时变的，为了提高编码器泛化能力，编码器被设计为自适应不同的信道条件。假设通道状态信息可用，我们将其作为参数添加到编码器。

B. Channel Model

编码后，压缩后的特征通过“不可靠”发送，这会被信道噪声破坏。这个过程如图 1 的底部方框所示。我们考虑两种不同类型的通道模型。首先我们考虑 AWGN 模型，传递函数写为 $f(x) = x + n$ ，其中 $n \sim \mathcal{N}(0, \sigma^2)$ 。参数 σ^2 捕获噪声方差作为通道条件。模拟通讯

在 AWGN 信道上采用这种方案，即压缩特征的符号将被直接调制，而不需要数字化或信道编码。此外，我们还考虑了二进制擦除通道 (BEC)，它也在 [16] 中用于基于 DNN 的通信系统。BEC 使用比特擦除率 (BER) p 来模拟通道模型中的深度衰落或突发错误，该模型具有二进制输入和三进制输出。在我们的实验中，擦除位的值设置为取 0 和 1 的位的平均值。例如，将二进制数转换为十进制数，110 的值为 6。但是，如果通道擦除最左边的位，则接收器会将 010 和 110 的平均值分配给它，即 4。对于 BEC 模型，连续数据在传输前必须量化为 n 位串。Sigmoid 函数的输出被限制在 $[0, 1]$ 中，因此量化器为 $\tilde{X} = \text{round}(X \cdot (2^n - 1)) / (2^n - 1)$ ，其中浮点 X 被量化为 $\tilde{X}$ ，由 n 位序列表示。

由于传递函数是可微的，AWGN 模型可以直接用于端到端训练。然而，BEC 的传递函数和量化是不可微的，阻碍了反向传播过程。为了解决这个问题，我们只需在反向传播中跳过通道模型。在实验中，我们发现忽略量化和通道损坏是可行的，并且当 p 的值不太大时模型仍然收敛。

C. Decoder

部署在接收器处的解码器是联合源通道解码器，用于将损坏的特征映射到恢复的特征。它由反卷积层、批量归一化层和 ReLU 激活函数组成。解码器的目的是将比特流恢复为压缩前相同维度的特征张量。反卷积层中的滤波器数量决定了输出通道数。为了从 C' 通道恢复 C 通道，反卷积层使用 C 滤波器。宽度方向和高度方向的恢复可以在卷积运算中采用 $\lfloor W/W' \rfloor, \lfloor H/H' \rfloor$ 步长，

where the tuple value is the same as the encoder. In our implementation, we set up the convolutional kernel size to 2×2 and stride to $(2, 2)$ in the encoder, which realize $2 \times$ width-compression and $2 \times$ height-compression, and the decoder also uses 2×2 size kernel and $(2, 2)$ stride. For both the encoder and decoder, we use the convolutional/deconvolutional network to compress the intermediate feature but not a fully connected layer. This is because, although the fully connected layer has stronger compression capability, its memory and computation cost is unacceptable. Compared to traditional coding algorithms, e.g., the LDPC code [18] and the Huffman code, our method enjoys a higher computational efficiency, and the encoding and decoding models are much simpler.

D. Training Strategy

As illustrated in Section III.B, the proposed BottleNet++ can be trained in an end-to-end manner with channel noise. However, directly training the whole architecture would suffer from the problem of slow convergence. Thus, we propose a three-step approach to train our end-to-end architecture. The first step is to train the DNN, e.g., VGG16 [19] or ResNet50 [9], to reach the desired accuracy of the task. A DNN consists of many layers, like the top of Fig. 1 excluding BottleNet++. The second step is to select the splitting point to deploy the BottleNet++, and then train and update the weights of the encoder and decoder while fixing other parameters in the DNN. In this step, training the encoder and decoder in different channel conditions, i.e., with different values of σ in the AWGN channel and p in the BEC, will benefit its generalization ability. In the last step, we fine-tune the whole network to increase the accuracy further. In this process, all the parameters in both the DNN and BottleNet++ are updatable, and we train the whole BottleNet++ with a low learning rate.

Compared with the architecture similar to our proposal, i.e., BottleNet [13], our method explicitly models the wireless channel as a non-trainable layer in DNN, while BottleNet assumes reliable communication over the wireless channel and ignores the bandwidth expansion caused by channel coding. As BottleNet++ considers channel conditions and exploits the fault-tolerance property of DNNs, the encoded bitstream does not need to be protected by a powerful channel encoder, and thus it enjoys a higher compression capability compared with BottleNet. For the training process, BottleNet adopts a compression-aware training approach, which trains the DNN and the compression module simultaneously. In contrast, our method adopts a three-step approach to train the whole architecture, which improves the convergence rate. Moreover, when we need to adjust the compression ratio to different channel conditions, we only need to retrain the compression module instead of the whole model.

IV. EVALUATION

A. Experimental Setup

We consider a classification task with the CIFAR-100 dataset [20] that consists of 60,000 32×32 color images in 100 classes, with 600 images per class. Since the edge devices

usually collect and deal with low-resolution images in IoT devices, CIFAR-100 is well suited for on-device applications. Several DNN architectures have been proposed to achieve outstanding classification performance, e.g., VGGNet [19], ResNet [9], DenseNet [21], etc. In this experiment, we use classical ResNet50 and VGG16. Although they can not achieve the start-of-the-art performance on CIFAR-100, our focus is on testing the compression capability of our method. In our experiment, VGG16 reaches 74.04% accuracy, and ResNet50 achieves 77.81% accuracy. The configuration of ResNet50 approximately follows [9], but, in the first convolutional layer, we change the kernel size to 3×3 and modify the stride of the convolutional operation to 1. For VGG16, its structure directly follows [19]. The loss function in the training process is the cross-entropy function.

As discussed in Section III.B, we consider the AWGN channel and BEC. The peak signal-to-noise ratio (PSNR) is used to indicate the AWGN channel condition, and the bit erasure rate (BER) is used to indicate the quality of a BEC. Because the maximum output of the encoder is 1, we define PSNR as:

$$\text{PSNR} = 10 \log_{10} \frac{1}{\sigma^2} \text{ (dB)} \quad (1)$$

To evaluate our proposed method, we split the DNN and compress the intermediate feature at different splitting points of VGG16 and ResNet50. Note that not all the layers in a DNN can be used as a splitting point. For sequential DNNs like VGG and AlexNet [22], input signals flow layer by layer, and we can easily split the network at the end of each layer. However, the latest deep models like ResNet introduce branchy network structures rather than sequential models. So, the splitting points are different. In our evaluation, each res-unit in ResNet [9] is regarded as a possible splitting point, while for VGG16, each convolutional layer can be regarded as a splitting point. The code is available at github.com/shaojiawei07/BottleNetPlusPlus.

B. Compression Capability Comparison

The latency of the device-edge co-inference system is mainly composed of on-device computation latency and transmission latency/communication overhead. So, we should split the network as early as possible (close to the input layer), with less on-device computation, and compress the intermediate feature as much as possible without too much loss of accuracy. In the experiment, the accuracy degradation threshold is set to 2%, and the on-device computation cost is approximated by the number of floating-point multiplication operations (FLOPs) in the convolutional layers before the splitting point.

In the following, we compare the compression capability of BottleNet++ with other methods, in both the BEC and AWGN channels, at different splitting points. Three baseline methods are considered: the method in [6] that adopts the JPEG algorithm for lossy compression, denoted as “JPEG”. The approach in [8] that quantizes floating-point data to n bit-depth and then encodes the result with Huffman coding denoted as “Quantization + Huffman”, and BottleNet [13]

其中元组值与编码器相同。在我们的实现中，我们在编码器中将卷积核大小设置为 2×2 ，步长设置为 (2, 2)，实现 $2 \times$ 宽度压缩和 $2 \times$ 高度压缩，解码器也使用 2×2 大小的核和 (2, 2) 步长。对于编码器和解码器，我们使用卷积/反卷积网络来压缩中间特征，而不是全连接层。这是因为，虽然全连接层具有更强的压缩能力，但其内存和计算成本却是不可接受的。与传统的编码算法（例如LDPC码[18]和Huffman码）相比，我们的方法具有更高的计算效率，并且编码和解码模型更简单。

D. Training Strategy

如第 III.B 节所示，所提出的 BottleNet++ 可以使用通道噪声以端到端的方式进行训练。然而，直接训练整个架构会遇到收敛速度慢的问题。因此，我们提出了一种三步方法来训练我们的端到端架构。第一步是训练 DNN，例如 VGG16 [19] 或 ResNet50 [9]，以达到任务所需的准确性。DNN 由许多层组成，如图 1 的顶部所示，不包括 BottleNet++。第二步是选择分裂点来部署 BottleNet++，然后训练和更新编码器和解码器的权重，同时固定 DNN 中的其他参数。在此步骤中，在不同的信道条件下训练编码器和解码器，即在 AWGN 信道中使用不同的 σ 值和在被 C 中使用 p 值，将有利于其泛化能力。在最后一步中，我们对整个网络进行微调以进一步提高准确性。在此过程中，DNN 和 BottleNet++ 中的所有参数都是可更新的，并且我们以较低的学习率训练整个 BottleNet++。

与我们提出的类似架构相比，即 BottleNet [13]，我们的方法明确地将无线信道建模为 DNN 中的不可训练层，而 BottleNet 假设无线信道上的可靠通信并忽略信道编码引起的带宽扩展。由于 BottleNet++ 考虑了通道条件并利用了 DNN 的容错特性，因此编码后的比特流不需要强大的通道编码器的保护，因此与 BottleNet 相比，它具有更高的压缩能力。对于训练过程，BottleNet 采用压缩感知训练方法，同时训练 DNN 和压缩模块。相比之下，我们的方法采用三步方法来训练整个架构，从而提高了收敛速度。此外，当我们需要根据不同的信道条件调整压缩比时，我们只需要重新训练压缩模块而不是整个模型。

四. 评估

A. Experimental Setup

我们考虑使用 CIFAR-100 数据集 [20] 进行分类任务，该数据集由 100 个类别的 60,000 张 32×32 彩色图像组成，每个类别有 600 张图像。由于边缘设备

CIFAR-100 通常在物联网设备中收集和处理低分辨率图像，非常适合设备上应用。为了实现出色的分类性能，已经提出了几种 DNN 架构，例如 VGGNet [19]、ResNet [9]、DenseNet [21] 等。在本实验中，我们使用经典的 ResNet50 和 VGG16。尽管它们无法在 CIFAR-100 上实现最先进的性能，但我们的重点是测试我们方法的压缩能力。在我们的实验中，VGG16 达到了 74.04% 的准确率，ResNet50 达到了 77.81% 的准确率。ResNet50 的配置大致遵循[9]，但是，在第一个卷积层中，我们将内核大小更改为 3×3 ，并将卷积运算的步幅修改为 1。对于 VGG16，其结构直接遵循[19]。训练过程中的损失函数就是交叉熵函数。

正如第 III.B 节中所讨论的，我们考虑 AWGN 信道和 BEC。峰值信噪比 (PSNR) 用于指示 AWGN 信道状况，比特擦除率 (BER) 用于指示 BEC 的质量。由于编码器的最大输出为 1，我们将 PSNR 定义为：

$$\text{PSNR} = 10 \log_{10} \frac{1}{\sigma^2} \text{ (dB)} \quad (1)$$

为了评估我们提出的方法，我们分割 DNN 并在 VGG16 和 ResNet50 的不同分割点压缩中间特征。请注意，并非 DNN 中的所有层都可以用作分割点。对于像 VGG 和 AlexNet [22] 这样的顺序 DNN，输入信号逐层流动，我们可以轻松地在每层末尾分割网络。然而，像 ResNet 这样的最新深度模型引入了分支网络结构而不是顺序模型。所以，分割点是不同的。在我们的评估中，ResNet [9] 中的每个 res-unit 都被视为一个可能的分割点，而对于 VGG16，每个卷积层都可以被视为一个分割点。代码可在 github.com/shaojiawei07/BottleNetPlusPlus 获取。

B. Compression Capability Comparison

设备边缘协同推理系统的延迟主要由设备端计算延迟和传输延迟/通信开销组成。因此，我们应该尽早分割网络（靠近输入层），减少设备上的计算，并尽可能地压缩中间特征，而不会损失太多的精度。在实验中，精度下降阈值设置为 2%，设备上的计算成本近似为分割点之前卷积层中的浮点乘法运算 (FLOP) 的数量。

下面，我们在 BEC 和 AWGN 通道中，在不同的分割点上比较 BottleNet++ 与其他方法的压缩能力。考虑三种基线方法：[6] 中的方法采用 JPEG 算法进行有损压缩，记为“JPEG”。[8] 中的方法将浮点数据量化为 n 位深度，然后使用表示为“量化 + Huffman”的霍夫曼编码和 BottleNet [13] 对结果进行编码

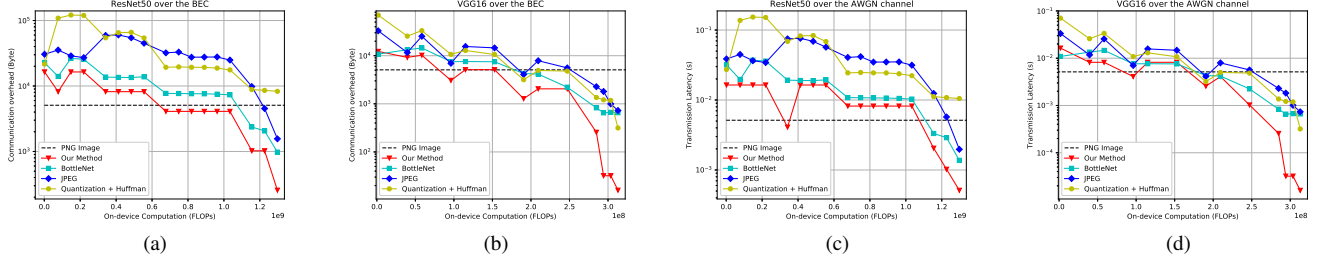


Fig. 2: On-device computation vs. communication overhead in BEC with (a) ResNet50 and (b) VGG16 and On-device computation vs. Transmission Latency in AWGN channel with (c) ResNet50 and (d) VGG16.

TABLE I: minimum on-device computation in different scenes with the communication overhead less than transmitting PNG image.

On-device Computation (FLOPs)	BottleNet++ (ours)	BottleNet	JPEG	Quan.+Huffman
ResNet50 (BEC)	6.8×10^8	1.1×10^9	1.2×10^9	1.3×10^9
ResNet50 (AWGN)	3.4×10^8	1.1×10^9	1.3×10^9	1.3×10^9
VGG16 (BEC)	9.6×10^7	1.9×10^8	1.9×10^8	1.9×10^8
VGG16 (AWGN)	9.6×10^7	1.9×10^8	1.9×10^8	1.9×10^8

that encodes the intermediate feature by a neural network and uses JPEG compression-aware training to reduce the accuracy loss. Furthermore, we consider the communication overhead of transmitting the raw PNG image from device to edge, denoted as “PNG image”. All of the baseline methods assumed reliable communication with necessary channel coding. In contrast, because our method integrates the channel model in the neural network, it can avoid extra cost for channel coding.

We first conduct the experiment over the BEC with the bit erasure rate $p = 0.01$. To ensure fairness, we apply the 1/2 rate LDPC code [18] to other methods as channel coding. Different code rates may affect the baseline curves in Fig. 2, but they do not change the result in Table I. Extensive experiment results show that even when the channel conditions and the code rates change, our BottleNet++ still performs better than other baseline methods and has similar results as Table I. The compression capability is represented by the communication overhead (size of the transmitted bitstream), which is equivalent to transmission latency. The result is shown in Fig. 2a and Fig. 2b and we note that almost at any splitting point, BottleNet++ achieves the lowest communication overhead. It realizes up to $256\times$ bit compression ratio in the last convolutional layer of ResNet, where BottleNet++ compresses 8192 32bit-floating numbers (32 KB) to 128 8bit-integers (128 Bytes).

Besides, with the requirement that the communication overhead of the intermediate feature should be less than that of raw PNG image, our BottleNet++ can split the network earlier than other methods. This result is summarized in Table I, which shows the on-device computation of different methods at the earliest splitting point. In any case, our BottleNet++ achieves the minimum on-device computation.

We next investigate the performance of compression capability over the AWGN channel. In this case, we directly

use the transmission latency as the indicator. Compared with baseline methods, BottleNet++ adopts the analog communication for transmitting the compressed feature, which bypasses quantization, source coding, and channel coding, and directly maps the data to continuous samples before transmission. We use quadrature modulation, and each of the I and Q channels carries an encoded but unquantized symbol. As the baseline methods assume perfect channel coding, we adopt the Shannon capacity formula to calculate their data rate, i.e., $C = W \log_2(1 + \text{SNR})$. Note that this gives a performance upper bound for the baseline methods, and thus in practice, the performance gain of BottleNet++ will be more prominent. The experiment fixes $W = 1$ MHz, $\text{SNR} = 14.5$ dB, and adopts transmission latency to evaluate the communication overhead. Specially, we set PSNR = 20 dB in BottleNet++ to satisfy its SNR equal to 14.5 dB, which is calculated from the transmitting power of each encoded symbol.

As shown in Fig. 2c and Fig. 2d, even assuming perfect coding for digital communications, the baseline methods introduce higher transmission latency than BottleNet++. In particular, BottleNet++ achieves up to $64\times$ bandwidth reduction in the second last convolutional layer of ResNet50, where it compresses the 2048-symbol feature to 32 symbols. Furthermore, similar to the BEC case, Table I shows that BottleNet++ reduces on-device computation by $\sim 2\times$ and $\sim 3\times$ for VGG16 and ResNet50 in the AWGN channel compared with other methods.

C. Generalization Ability and Robustness Analysis

In real communication systems, the channel condition is time-varying, and in this part, we test the generalization ability, i.e., accuracy degradation, of BottleNet++ when the channel condition changes. We conduct the experiment based on the ResNet50 model deployed behind the last convolutional layer, with $64\times$ compression ratio for the AWGN channel and with

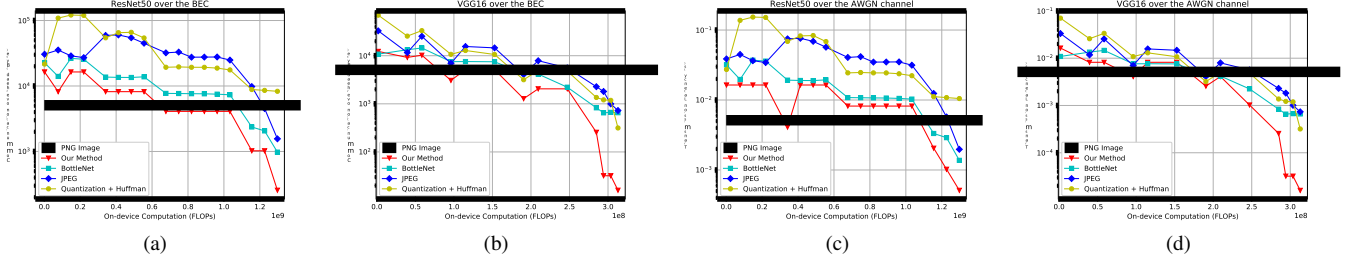


图 2: 使用 (a) ResNet50 和 (b) VGG16 的 BEC 中的设备上计算与通信开销以及使用 (c) ResNet50 和 (d) VGG16 的 AWGN 通道中的设备上计算与传输延迟。

表一: 不同场景下的最小设备上计算, 通信开销小于传输 PNG 图像。

On-device Computation (FLOPs)	BottleNet++ (ours)	BottleNet	JPEG	Quan.+Huffman
ResNet50 (BEC)	6.8×10^8	1.1×10^9	1.2×10^9	1.3×10^9
ResNet50 (AWGN)	3.4×10^8	1.1×10^9	1.3×10^9	1.3×10^9
VGG16 (BEC)	9.6×10^7	1.9×10^8	1.9×10^8	1.9×10^8
VGG16 (AWGN)	9.6×10^7	1.9×10^8	1.9×10^8	1.9×10^8

通过神经网络对中间特征进行编码, 并使用 JPEG 压缩感知训练来减少准确性损失。此外, 我们还考虑将原始 PNG 图像从设备传输到边缘的通信开销, 表示为“PNG 图像”。所有基线方法都假设具有必要的信道编码的可靠通信。相反, 由于我们的方法将信道模型集成到神经网络中, 因此可以避免信道编码的额外成本。

我们首先在比特擦除率 $p = 0.01$ 的 BEC 上进行实验。为了确保公平性, 我们将 1/2 速率 LDPC 码 [18] 应用于其他方法作为信道编码。不同的码率可能会影响图 2 中的基线曲线, 但不会改变表 1 中的结果。大量的实验结果表明, 即使信道条件和码率发生变化, 我们的 BottleNet++ 仍然比其他基线方法表现更好, 并且具有与表 1 类似的结果。压缩能力由通信开销 (传输比特流的大小) 表示, 这相当于传输延迟。结果如图 2a 和图 2b 所示, 我们注意到几乎在任何分裂点, BottleNet++ 都实现了最低的通信开销。它在 ResNet 的最后一个卷积层中实现了高达 $256\times$ 位的压缩比, 其中 BottleNet++ 将 8192 个 32 位浮点数字 (32 KB) 压缩为 128 个 8 位整数 (128 字节)。

此外, 由于要求中间特征的通信开销应小于原始 PNG 图像的通信开销, 因此我们的 BottleNet++ 可以比其他方法更早地分割网络。表 1 总结了这个问题, 它显示了不同方法在最早分裂点的设备上计算。无论如何, 我们的 BottleNet++ 实现了最小的设备上计算。

接下来我们研究 AWGN 信道上的压缩能力的性能。在这种情况下, 我们直接

以传输延迟为指标。与基线方法相比, BottleNet++ 采用模拟通信来传输压缩特征, 绕过了量化、信源编码和通道编码, 在传输前直接将数据映射为连续样本。我们使用正交调制, 每个 I 和 Q 通道都携带一个已编码但未量化的符号。由于基线方法假设完美的信道编码, 因此我们采用香农容量公式来计算其数据速率, 即 $C = W \log_2(1 + \text{SNR})$ 。请注意, 这给出了基线方法的性能上限, 因此在实践中, BottleNet++ 的性能增益将更加突出。实验固定 $W = 1 \text{ MHz}$, $\text{SNR} = 14.5 \text{ dB}$, 并采用传输延迟来评估通信开销。特别地, 我们将 BottleNet++ 中的 PSNR = 设置为 20 dB, 以满足其 SNR 等于 14.5 dB, 这是根据每个编码符号的发射功率计算得出的。

如图 2c 和图 2d 所示, 即使假设数字通信具有完美的编码, 基线方法也会带来比 BottleNet++ 更高的传输延迟。特别是, BottleNet++ 在 ResNet50 的倒数第二个卷积层中实现了高达 $64\times$ 的带宽减少, 它将 2048 个符号特征压缩为 32 个符号。此外, 与 BEC 情况类似, 表 1 显示, 与其他方法相比, 对于 AWGN 通道中的 VGG16 和 ResNet50, BottleNet++ 减少了 $\sim 2\times$ 和 $\sim 3\times$ 的设备上计算量。

C. Generalization Ability and Robustness Analysis

在实际通信系统中, 信道条件是时变的, 在这一部分中, 我们测试了 BottleNet++ 在信道条件变化时的泛化能力, 即精度下降。我们基于部署在最后一个卷积层后面的 ResNet50 模型进行实验, AWGN 通道的压缩比为 $64\times$, 并且

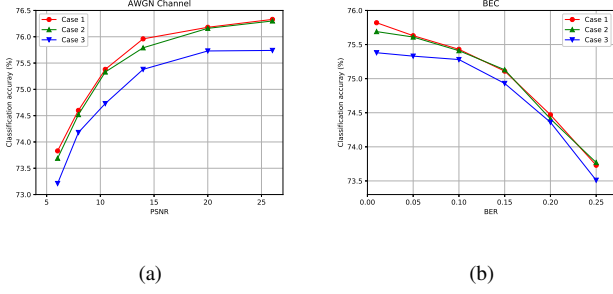


Fig. 3: Accuracy degradation of BottleNet++ over the (a) AWGN channel and (b) BEC.

256 $\times$ bit compression ratio for BEC, respectively. We evaluate the performance of BottleNet++ in three different cases:

- Case 1: The encoder knows the channel state information, i.e., the bit erasure rate or PSNR, in both the training and testing processes.
- Case 2: The encoder only knows the channel state information in the training process. In the testing process, the encoder assumes the channel condition to be 15 dB in the AWGN channel or 0.125 in the BEC.
- Case 3: The encoder does not know the channel state information in either the training or the testing process.

Fig. 3a and Fig. 3b presents the accuracy loss in different channel conditions of the three cases. Case 1 achieves the highest accuracy under any channel condition. Case 2 is very close to case 1, which shows the robustness of BottleNet++ to the variation of channel conditions. Case 3 has a noticeable accuracy drop compared to Case 1, which means that considering channel conditions during encoding can improve the generalization ability. Remarkably, the performance of BottleNet++ is robust to channel variations in all three cases. Specifically, the accuracy drops less than 1% when PSNR changes from 25 dB to 10 dB in the AWGN channel, or when the bit erasure rate changes from 0.01 to 0.15 in BEC.

V. CONCLUSIONS

In this paper, we proposed an end-to-end deep learning architecture, named BottleNet++, for device-edge co-inference with resource-constrained mobile devices. By exploiting the strong sparsity and the fault-tolerant property of the intermediate feature in the DNNs, BottleNet++ achieves a much higher compression ratio than existing methods, which leads to a significant reduction in the communication overhead, and makes it feasible to split a DNN in the earlier layer to reduce the on-device computation. For the communication theoretic aspect, our study casts new light on the two fundamental problems in the setting of device-edge co-inference: *What to transmit? How to transmit?* The results indicate that transmitting highly compressed features with analog communication becomes attractive for edge-assisted inference. Furthermore, DNNs stand out as powerful design tools for such new communication problems.

REFERENCES

- [1] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Region-based convolutional networks for accurate object detection and segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 1, pp. 142–158, 2015.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [3] K. Skala, D. Davidovic, E. Afgan, I. Sovic, and Z. Sojat, "Scalable distributed computing hierarchy: Cloud, fog and dew computing," *Open Journal of Cloud Computing (OJCC)*, vol. 2, no. 1, pp. 16–24, 2015.
- [4] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "Model compression and acceleration for deep neural networks: The principles, progress, and challenges," *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 126–136, 2018.
- [5] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "Jointdnn: an efficient training and inference engine for intelligent mobile cloud computing services," *arXiv preprint arXiv:1801.08618*, 2018.
- [6] J. H. Ko, T. Na, M. F. Amir, and S. Mukhopadhyay, "Edge-host partitioning of deep neural networks with feature space encoding for resource-constrained internet-of-things platforms," in *2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2018, pp. 1–6.
- [7] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1. ACM, 2017, pp. 615–629.
- [8] H. Li, C. Hu, J. Jiang, Z. Wang, Y. Wen, and W. Zhu, "Jalad: Joint accuracy and latency-aware deep structure decoupling for edge-cloud execution," in *2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS)*, 2018, pp. 671–678.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [10] E. Boursoulatz, D. B. Kurka, and D. Gündüz, "Deep joint source-channel coding for wireless image transmission," *IEEE Transactions on Cognitive Communications and Networking*, 2019.
- [11] H. Choi and I. V. Bajić, "Near-lossless deep feature compression for collaborative intelligence," in *2018 IEEE 20th International Workshop on Multimedia Signal Processing (MMSP)*. IEEE, 2018, pp. 1–6.
- [12] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang, and L. Geng, "Improving device-edge cooperative inference of deep learning via 2-step pruning," *arXiv preprint arXiv:1903.03472*, 2019.
- [13] A. E. Eshratifar, A. Esmaili, and M. Pedram, "Bottlenet: A deep learning architecture for intelligent mobile cloud computing services," *arXiv preprint arXiv:1902.01000*, 2019.
- [14] C. E. Shannon, "A mathematical theory of communication," *Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [15] F. Zhai, Y. Eisenberg, and A. K. Katsaggelos, "Joint source-channel coding for video communications," *Handbook of Image and Video Processing*, 2005.
- [16] K. Choi, K. Tatwawadi, A. Grover, T. Weissman, and S. Ermon, "Neural joint source-channel coding," in *International Conference on Machine Learning*, 2019, pp. 1182–1192.
- [17] M. D. Emmerson and R. I. Damper, "Determining and improving the fault tolerance of multilayer perceptrons in a pattern-recognition application," *IEEE Transactions on Neural Networks*, vol. 4, no. 5, pp. 788–793, 1993.
- [18] A. Shokrollahi, "Ldpc codes: An introduction," *Digital Fountain, Inc., Tech. Rep.*, vol. 2, p. 17, 2003.
- [19] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations*, 2015.
- [20] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.
- [21] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

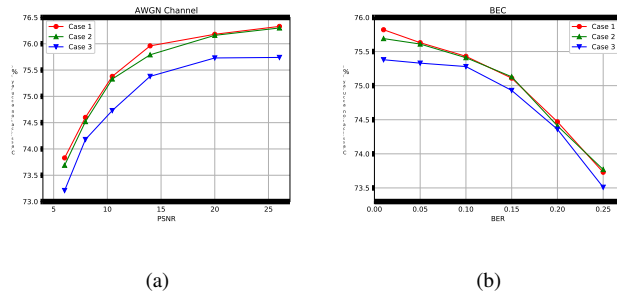


图 3: BottleNet++ 在 (a) AWGN 通道和 (b) BEC 上的精度下降。

BEC 的压缩比分别为 $256 \times$ 位。我们在三种不同情况下评估 BottleNet++ 的性能:

- 情况 1: 编码器在训练和测试过程中都知道信道状态信息, 即比特擦除率或 PSNR。
- 情况 2: 编码器在训练过程中只知道信道状态信息。在测试过程中, 编码器假设信道条件为 AWGN 信道中的 15 dB 或 BEC 中的 0.125。
- 情况 3: 编码器在训练或测试过程中都不知道信道状态信息。

图 3a 和图 3b 展示了三种情况下不同信道条件下的精度损失。情况 1 在任何信道条件下都达到了最高的精度。情况 2 与情况 1 非常接近, 这显示了 BottleNet++ 对通道条件变化的鲁棒性。与情况 1 相比, 情况 3 的准确率明显下降, 这意味着在编码过程中考虑信道条件可以提高泛化能力。值得注意的是, BottleNet++ 的性能在所有三种情况下都对通道变化具有鲁棒性。具体来说, 当 AWGN 信道中 PSNR 从 25 dB 变化到 10 dB 时, 或者当 BEC 中比特擦除率从 0.01 变化到 0.15 时, 精度下降不到 1%。

五、结论

在本文中, 我们提出了一种名为 BottleNet++ 的端到端深度学习架构, 用于与资源受限的移动设备进行设备边缘协同推理。通过利用 DNN 中中间特征的强稀疏性和容错特性, BottleNet++ 实现了比现有方法更高的压缩比, 从而显着减少了通信开销, 并且使得在较早的层中分割 DNN 以减少设备上的计算成为可能。对于通信理论方面, 我们的研究为设备边缘协同推理设置中的两个基本问题提供了新的思路: *What to transmit?* *How to transmit?* 结果表明, 通过模拟通信传输高度压缩的特征对于边缘辅助推理变得有吸引力。此外, DNN 作为解决此类新通信问题的强大设计工具脱颖而出。

- [1] R. Girshick, J. Donahue, T. Darrell and J. Malik, “用于精确对象检测和分割的基于区域的卷积网络”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 卷. 38, 没有. 1, 第 142–158 页, 2015 年。[2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari and M. Ayyash, “物联网: 关于支持技术、协议和应用程序的调查”, *IEEE Communications Surveys & tutorials*, 卷. 17, 没有. 4, 第 2347–2376 页, 2015 年。[3] K. Skala, D. Davidovic, E. Afgan, I. Sovic 和 Z. Sojat, “可扩展的分布式计算层次结构: 云、雾和露水计算”, *Open Journal of Cloud Computing (OJCC)*, 卷. 2, 没有. 1, 第 16–24 页, 2015 年。[4] Y. Cheng, D. Wang, P. Zhou 和 T. Zhang, “深度神经网络的模型压缩和加速: 原理、进展和挑战”, *IEEE Signal Processing Magazine*, 卷. 35, 没有. 1, 第 126–136 页, 2018 年。[5] A. E. Eshratifar, M. S. Abrishami 和 M. Pedram, “Jointdnn: 智能移动云计算服务的高效训练和推理引擎”, *arXiv preprint arXiv:1801.08618*, 2018 年。[6] J. H. Ko, T. Na, M. F. Amir 和 S. Mukhopadhyay, “针对资源受限的物联网平台使用特征空间编码进行深度神经网络的边缘主机分区”, *2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2018 年, 第 1–6 页。[7] Y. Kang, J. Hauswald, C. Gau, A. Rovinski, T. Mudge, J. Mars 和 L. Tang, “神经外科医生: 云和移动边缘之间的协作智能”, 第 *ACM SIGARCH Computer Architecture News* 卷. 45, 没有. 1. ACM, 2017 年, 第 615–629 页。[8] H. Li, C. Hu, J. Jiang, Z. Wang, Y. Wen 和 W. Zhu, “Jalad: 边缘云执行的联合精度和延迟感知深度结构解耦”, *2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS)*, 2018 年, 第 671–678 页。[9] K. He, X. Zhang, S. Ren, J. Sun, “图像识别的深度残差学习”, 载于 *Proc. the IEEE conference on computer vision and pattern recognition*, 2016 年, 第 770–778 页。[10] E. Boursoulatz, D. B. Kurka 和 D. Gündüz, “用于无线图像传输的深度联合源信道编码”, *IEEE Transactions on Cognitive Communications and Networking*, 2019。[11] H. Choi 和 I. V. Bajić, “用于协作智能的近无损深度特征压缩”, *2018 IEEE 20th International Workshop on Multimedia Signal Processing (MMSP)*. IEEE, 2018 年, 第 1–6 页。[12] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang 和 L. Geng, “通过两步剪枝改进深度学习的设备边缘协作推理”, *arXiv preprint arXiv:1903.03472*, 2019。[13] A. E. Eshratifar, A. Esmaili 和 M. Pedram, “Bottlenet: 智能移动云计算服务的深度学习架构”, *arXiv preprint arXiv:1902.01000*, 2019。[14] C. E. Shannon, “通信的数学理论”, *Bell system technical journal*, 卷. 27, 没有. 3, 第 379–423 页, 1948 年。[15] F. Zhai, Y. Eisenberg 和 A. K. Katsaggelos, “视频通信的联合源通道编码”, *Handbook of Image and Video Processing*, 2005 年。[16] K. Choi, K. Tatwawadi, A. Grover, T. Weissman 和 S. Ermon, “神经联合源通道编码”, 载于 *International Conference on Machine Learning*, 2019 年, 第 1182–1192 页。[17] M. D. Emmerson 和 R. I. Damper, “确定和提高模式识别应用中多层感知器的容错能力”, *IEEE Transactions on Neural Networks*, 卷. 4, 没有. 5, 第 788–793 页, 1993 年。[18] A. Shokrollahi, “Ldpc 代码: 简介”, *Digital Fountain, Inc., Tech. Rep.*, 卷. 2, 第 14 页。2003 年 17 月。[19] K. Simonyan 和 A. Zisserman, “用于大规模图像识别的非常深的卷积网络”, *International Conference on Learning Representations*, 2015 年。[20] A. Krizhevsky, G. Hinton et al., “从微小图像中学习多层特征”, *Citeseer, Tech. Rep.*, 2009。[21] G. Huang, Z. Liu, L. Van Der Maaten 和 K. Q. Weinberger, “密集连接的卷积网络”, 载于 *Proc. the IEEE conference on computer vision and pattern recognition*, 2017 年, 第 4700–4708 页。[22] A. Krizhevsky, I. Sutskever 和 G. E. Hinton, “使用深度卷积神经网络进行 Imagenet 分类”, 载于 *Advances in neural information processing systems*, 2012 年, 第 1097–1105 页。